CSCI 6461 | TOPIC 2

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA choices, implementation costs, and performance consequences

Dr. J. Burns

Computer Systems Architecture

What Is an Instruction Set Architecture?

- An ISA is the programmer-visible specification of a processor.
- It defines instructions, registers, data types, addressing modes, and architectural state.
- Compilers generate code for the ISA; processors implement the ISA.
- ISA design therefore sits directly at the hardware/software boundary.
- A good ISA must remain useful for decades while implementations evolve underneath it.
- RISC and CISC are different historical answers to the question: **what should the ISA expose?**

Why ISA Design Matters

Software consequences

- Instruction count
- Code size
- Compiler complexity
- Calling conventions
- Binary compatibility

Hardware consequences

- Decode complexity
- Datapath organization
- Pipeline regularity
- Control logic
- Energy per instruction

ISA design redistributes complexity between software and hardware rather than eliminating it.

The Instruction-Set Design Problem

- Should one instruction perform a small primitive operation or a larger sequence of work?
- Should arithmetic instructions access memory directly or operate only on registers?
- Should instructions have one fixed length or many lengths?
- How many addressing modes should hardware understand?
- Should frequently used operations receive special instructions even if they complicate decode?
- RISC and CISC emerged from different answers shaped by the technology of their eras.

Historical Constraints Shaped Early ISAs

- Early main memory was expensive, slow, and physically limited.
- Compilers were less sophisticated than modern optimizing compilers.
- Hardware designers wanted programs to occupy fewer bytes and make fewer memory accesses.
- Rich instructions could express substantial work in a compact encoding.
- Microprogrammed control allowed complex instructions to be implemented as sequences of simpler internal operations.
- These pressures strongly encouraged what later became known as the CISC style.

The Origins of CISC

- CISC stands for **Complex Instruction Set Computer**.
- The philosophy favored rich instruction sets with many operations and addressing modes.
- Individual instructions could perform multi-step tasks such as memory access plus arithmetic.
- Variable-length encodings allowed common instructions to be compact while retaining expressive instructions.
- The goal was often to reduce program size and narrow the semantic gap between languages and machine code.
- The x86 ISA is the most prominent surviving example of a CISC lineage.

The CISC Design Philosophy

- Provide many instructions so software can request specialized operations directly.
- Permit operands to reside in registers or memory in many instruction forms.
- Support multiple addressing modes to make data structures convenient to access.
- Use variable instruction lengths to encode simple cases compactly and complex cases when needed.
- Allow one architectural instruction to correspond to several internal hardware actions.
- Accept more complicated decoding and control in exchange for expressive machine instructions.

CISC Example: More Work per Instruction

Conceptual example

Imagine an ISA instruction that computes `a = a + memory[b + 8]`.

- One architectural instruction identifies an address, reads memory, performs arithmetic, and writes a result.
- The instruction may encode an operation, base register, displacement, and destination.
- Fewer instructions are visible in the program.
- Internally, hardware may still perform address generation, memory access, addition, and write-back separately.
- The architectural instruction is compact semantically even if its implementation is multi-step.
- This distinction between **architectural instruction** and **internal work** is important.

Why Complex Instructions Can Be Attractive

- Fewer instructions can reduce instruction-fetch traffic.
- Rich addressing modes can express common data-access patterns directly.
- Compact encodings improve code density, which can benefit instruction-cache behavior.
- Some specialized operations map naturally to common software idioms.
- Backward compatibility can preserve huge software ecosystems over many processor generations.
- The attraction is not complexity for its own sake, but accomplishing useful work with fewer architectural instructions or bytes.

The Cost of Instruction Complexity

- Variable instruction length makes instruction boundaries harder to find quickly.
- Many addressing modes increase decoder and control complexity.
- Instructions may require very different execution times and hardware resources.
- Irregular operand rules complicate pipelines and dependency handling.
- Complex decode consumes area and energy before useful execution begins.
- These costs became increasingly important as processors became deeply pipelined.

The Emergence of RISC

- Researchers observed that compilers used only a subset of many complex instruction sets frequently.
- Simple operations often dominated real compiled workloads.
- Semiconductor integration made larger register files increasingly practical.
- Faster pipelines rewarded instructions with predictable execution behavior.
- Compiler technology improved enough to manage more of the scheduling and instruction-selection burden.
- These observations motivated Reduced Instruction Set Computer designs.

The RISC Design Philosophy

- Favor regular instructions that are straightforward to fetch, decode, and execute.
- Perform arithmetic primarily on registers.
- Access memory explicitly through load and store instructions.
- Use relatively regular instruction encodings.
- Provide enough registers to keep active values close to the execution units.
- Let compilers combine simple operations into efficient instruction sequences.

Berkeley RISC and Stanford MIPS

- RISC research accelerated during the late 1970s and early 1980s.
- Berkeley RISC demonstrated the value of simple instructions and large register sets.
- Stanford MIPS emphasized pipeline-friendly instruction behavior.
- Both projects challenged the assumption that increasingly complex ISAs were necessary for performance.
- Their ideas strongly influenced later commercial processor families.
- ARM emerged in the same broader movement toward simpler, regular instruction-set design.

RISC Principle: Simple and Regular Instructions

- Regularity makes instruction behavior easier for hardware to predict.
- Similar instruction formats simplify field extraction during decode.
- Operations tend to have clearly defined sources and destinations.
- Simpler instruction semantics reduce special-case control paths.
- Regularity becomes particularly valuable when many instructions overlap in a pipeline.
- The goal is not necessarily fewer instruction types, but predictable structure.

RISC Principle: Load/Store Architecture

- Arithmetic and logical instructions operate primarily on registers.
- Loads transfer data from memory into registers.
- Stores transfer data from registers back to memory.
- Arithmetic instructions therefore do not normally need to wait directly for arbitrary memory operands.
- Memory behavior becomes concentrated in explicit instruction classes.
- AArch64 follows this load/store organization.

RISC Principle: Register-Based Computation

- Registers are much faster to access than main memory.
- A large register set allows temporary values to remain near execution hardware.
- Compilers attempt to assign frequently used program values to registers.
- Register-to-register operations have regular operand access behavior.
- Reduced memory traffic can improve both performance and energy efficiency.
- Register allocation therefore becomes an important compiler responsibility.

RISC Principle: Regular Instruction Encoding

- Fixed-width instructions make instruction boundaries immediately known.
- Fetch hardware can retrieve several aligned instructions predictably.
- Decoder logic knows where common fields are likely to appear.
- Parallel decoding becomes easier when instruction boundaries are regular.
- AArch64 uses fixed 32-bit instruction encodings.
- Encoding regularity is one reason RISC designs map naturally onto pipelines.

RISC Principle: Compiler-Friendly ISA Design

- RISC deliberately exposes relatively primitive operations to software.
- Compilers decide how those operations should be combined.
- Register allocation determines which values remain in fast registers.
- Instruction scheduling can rearrange independent operations to reduce stalls.
- A regular ISA gives the compiler predictable building blocks.
- Some complexity therefore moves from instruction semantics into compiler optimization.

Worked Example: Evaluating an Expression in RISC

Suppose:

$$c = a + b$$

and the values are stored in memory.

Conceptual AArch64-style sequence

```
ldr x1, [x0]
ldr x2, [x0, #8]
add x3, x1, x2
str x3, [x0, #16]
```

- Memory access and arithmetic are separate operations.
- The actual addition occurs between register operands.
- Four architectural instructions are visible.

Hypothetical CISC Equivalent

The same conceptual operation might be exposed as:

Hypothetical CISC instruction

```
add_mem [c], [a], [b]
```

Conceptually, this one instruction performs:

1. Read operand `a` from memory.
2. Read operand `b` from memory.
3. Add the two values.
4. Write the result to memory location `c`.

Important distinction

This is a deliberately hypothetical instruction used to illustrate CISC philosophy; it is not being presented as an actual x86 instruction.

RISC vs. CISC: Same Computation, Different Interface

RISC view

Several explicit operations:

- load
- load
- add
- store

CISC view

One richer architectural operation may:

- identify memory operands
- perform address generation
- perform arithmetic
- update memory

The CISC sequence may contain fewer architectural instructions, but that does not imply less internal processor work.

Instruction Count: Is Fewer Always Better?

- A complex instruction can replace several simpler instructions.
- This may reduce dynamic instruction count.
- Fewer instructions can also reduce instruction-fetch bandwidth.
- But instruction count is only one term in processor performance.
- A complex instruction may require more cycles or more decoding work.
- Therefore lower instruction count does not automatically imply lower execution time.

Cycles per Instruction: The Other Side

- CPI measures average clock cycles consumed per executed instruction.
- Simple instructions may be easier to execute with predictable latency.
- Complex instructions may require several internal operations.
- Modern superscalar processors can execute multiple simple operations concurrently.
- Memory delays can dominate the CPI of either ISA style.
- Instruction count and CPI must therefore be considered together.

Clock Cycle Time and Instruction Complexity

- Processor clock period is constrained by critical hardware paths.
- More complicated decode and control can increase front-end delay.
- Simpler instruction formats can make high-speed decoding easier.
- Modern processors use pipelining to divide complex work across stages.
- A longer pipeline can support higher clock frequency but introduces other penalties.
- ISA complexity can therefore influence clock design indirectly through implementation choices.

The Iron Law Applied to ISA Design

$$\boxed{T_{CPU} = IC \times CPI \times T_{cycle}}$$

- CISC may reduce **instruction count**.
- RISC regularity may help reduce **CPI**.
- Simpler decode and pipeline structures may help reduce **clock cycle time**.
- These effects can oppose one another.
- No single term determines performance by itself.
- ISA design must therefore be evaluated using complete workload execution.

Worked Performance Comparison

Assume two implementations execute the same workload.

	IC	CPI	Clock Rate
RISC	1.4×10^9	1.1	3.0 GHz
CISC	1.0×10^9	1.6	2.8 GHz

$$T_{\text{RISC}} = \frac{1.4 \times 10^9 \times 1.1}{3.0 \times 10^9} \approx 0.513 \text{ s}$$

$$T_{\text{CISC}} = \frac{1.0 \times 10^9 \times 1.6}{2.8 \times 10^9} \approx 0.571 \text{ s}$$

The RISC machine executes more instructions yet completes the workload sooner.

Code Density and Memory Footprint

- Program size depends on both instruction count and bytes per instruction.
- Variable-length CISC encodings can represent common operations compactly.
- Fixed-width RISC instructions may require more bytes for some sequences.
- Higher code density can improve instruction-cache utilization.
- Better instruction-cache behavior can reduce memory traffic and energy.
- Code density therefore remains a genuine architectural advantage, especially when memory capacity or bandwidth is constrained.

Instruction Fetch and Decode Complexity

Regular fixed-width encoding

- boundaries known immediately
- easier parallel fetch
- regular field extraction
- simpler alignment

Variable-length encoding

- boundaries must be discovered
- instructions may cross fetch boundaries
- decode may require more stages
- more front-end hardware may be required

Modern CISC processors invest substantial hardware in making this front end fast enough to feed wide execution engines.

ISA Design and Pipelining

- Pipelines work best when instruction behavior can be divided into regular stages.
- RISC load/store organization separates memory operations from arithmetic operations.
- Fixed instruction width simplifies fetch and decode stage boundaries.
- Similar operand structures make hazard detection easier to organize.
- Irregular instructions can still be pipelined, but may require additional translation or control.
- RISC historically aligned well with the needs of deeply pipelined processors.

ISA Design and Compiler Optimization

- Compilers translate high-level program behavior into ISA operations.
- A regular instruction set provides predictable optimization targets.
- Register allocation determines which program values remain in registers.
- Instruction scheduling attempts to expose independent operations.
- Rich CISC instructions can sometimes express useful operations directly.
- ISA quality therefore includes how effectively real compiler toolchains can exploit its features.

RISC vs. CISC: Energy and Thermal Trade-offs

RISC tendency

- Regular encodings can simplify fetch, alignment, and decode.
- Simpler architectural operations can reduce front-end work per instruction.
- Explicit load/store behavior can make data movement more predictable.

CISC tendency

- Variable-length or richer instructions can require more complex decode.
- One instruction may perform several operations or memory accesses.
- Better code density can reduce instruction-fetch traffic.

Architectural implication

ISA style influences energy, but implementation and workload determine total efficiency.

Energy per Instruction Is Not Energy per Program

A useful first-order model

$$E_{\text{program}} \approx IC \times E_{\text{instruction}}$$

- A simpler instruction may consume less energy when fetched, decoded, and executed.
- But a RISC sequence may require more architectural instructions to complete the same task.
- A denser CISC sequence may reduce instruction-cache accesses.
- Memory behavior can dominate energy when cache misses reach DRAM.
- The relevant metric is total energy for the workload.
- This mirrors the Iron Law: optimizing one factor alone can produce the wrong conclusion.

Power Becomes Heat — and Heat Requires Cooling

Energy and power

$$P = \frac{E}{t}$$

Higher switching activity, voltage, frequency, and active hardware structures increase power demand.

System consequence

architectural complexity

↓

transistor switching

↓

power consumption

↓

heat generation

↓

cooling requirement

- Cooling ranges from passive heat spreaders to fans, liquid cooling, and data-center HVAC.
- Thermal limits can force frequency reduction, so power can ultimately constrain performance.

Performance per Watt: The More Useful Question

- Processor design increasingly optimizes useful work per unit of energy.
- Simpler decode and regular execution can contribute to efficiency.
- Code density, caches, speculation, vector units, and memory traffic can reverse simple ISA-level expectations.
- Process technology, voltage, frequency, and microarchitecture often matter more than the RISC/CISC label.
- ARM's success now spans phones, laptops, and cloud servers.
- Therefore, "ARM is efficient because RISC is simpler" is an incomplete architectural explanation.

Architectural implication

Compare **energy per workload**, **performance per watt**, and **thermal constraints**; ISA labels are only part of the evidence.

Why Modern x86 Looks RISC-Like Inside

- Modern x86 cores preserve the complex x86 ISA for software compatibility.
- The front end decodes many x86 instructions into simpler internal micro-operations (μops).
- Those μops are scheduled onto regular execution resources.
- This separates a complex architectural interface from a more regular internal execution model.
- Some x86 instructions decode into one μop; others require several.
- ISA style and microarchitecture style are therefore not the same thing.

Why Modern RISC ISAs Are Not Necessarily "Simple"

- Modern RISC ISAs include SIMD, cryptography, virtualization, atomics, and system-control instructions.
- AArch64 contains many instruction classes and sophisticated addressing features.
- High-performance ARM cores use branch prediction, out-of-order execution, speculation, and large caches.
- None of this violates the central RISC ideas of regular encoding and load/store organization.
- "Reduced" should be understood historically as architectural regularity, not a tiny instruction count.
- Calling a modern ARM core "simple" confuses ISA philosophy with implementation sophistication.

RISC Beyond Phones: Laptops

- Modern RISC is no longer confined to embedded controllers or phones.
- Apple Silicon brought Arm-based processors into mainstream laptop and desktop computing.
- Current MacBook Pro systems use Apple M5-family processors.
- These systems target high performance while retaining strong energy efficiency and battery life.
- Laptops therefore demonstrate the importance of performance per watt, not merely low power.
- RISC principles can scale from constrained devices to high-performance personal computers.

RISC in the Data Center: Google Axion

- Cloud providers now deploy Arm-based processors for general-purpose server workloads.
- Google Axion is a custom Arm-based CPU designed for Google Cloud data-center workloads.
- Target workloads include web servers, databases, analytics, media processing, and CPU-based AI.
- Google has described deployment for Bigtable, Spanner, BigQuery, Blobstore, Pub/Sub, Earth Engine, and YouTube Ads.
- These are latency-sensitive and throughput-oriented production workloads.
- At data-center scale, performance per watt affects server density, electricity cost, heat generation, and cooling infrastructure.

Data-Center Workloads: What Runs on Arm?

Storage and databases

- Spanner
- Bigtable
- Cloud SQL / AlloyDB-class workloads

Analytics and pipelines

- BigQuery
- Dataflow-style pipelines
- Batch and containerized jobs

Serving and platforms

- YouTube Ads
- Web and application servers
- Containerized microservices

AI and inference

- CPU-based inference
- Preprocessing and data movement
- Non-accelerator-dominated services

ARM AArch64 as a Modern RISC ISA

- AArch64 uses fixed 32-bit instruction encodings.
- Most arithmetic and logical operations use register operands.
- Memory is accessed primarily through explicit load and store instructions.
- Thirty-one general-purpose integer registers support register-oriented computation.
- Instruction formats are structured so common fields remain relatively regular.
- These properties make AArch64 useful for studying the connection between ISA and datapath design.

Next topic

The course now moves from ISA design philosophy to the actual AArch64 programmer's model and assembly language.

RISC vs. CISC: What Actually Matters Today?

- The RISC/CISC labels are less predictive than they were in the 1980s.
- Modern CISC processors use sophisticated internal decomposition and execution engines.
- Modern RISC processors contain complex microarchitectures and rich ISA extensions.
- Compatibility, code density, compiler ecosystem, implementation quality, and workload matter enormously.
- The enduring RISC contribution is emphasis on regularity, load/store organization, and compiler-visible simple operations.
- Architectural choices should be evaluated quantitatively rather than through slogans.

Synthesis: ISA Choices Are Trade-offs

- CISC tends to expose richer operations and often achieves strong code density.
- RISC tends to expose more regular operations that are easier to decode and pipeline.
- Neither lower instruction count nor higher clock rate guarantees better performance.
- The Iron Law requires instruction count, CPI, and cycle time to be considered together.
- Energy, heat, cooling, and performance per watt extend the analysis beyond execution time alone.
- AArch64 provides a modern RISC case study for the remainder of the course.

Design question

Do not ask "Which philosophy is faster?"

Ask: **What trade-off does this ISA choice create, and what does the measured workload tell us?**